自然語言處理與應用
Natural Language Processing and Applications

注意力機制與現代字詞分割演算法

Instructor: 林英嘉 (Ying-Jia Lin)

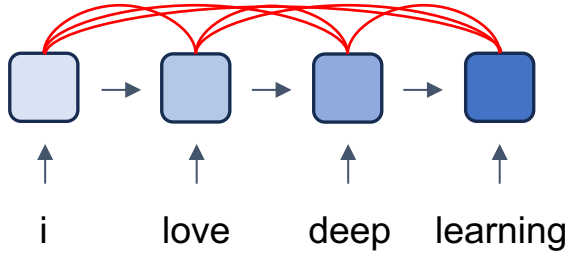
2026/03/16

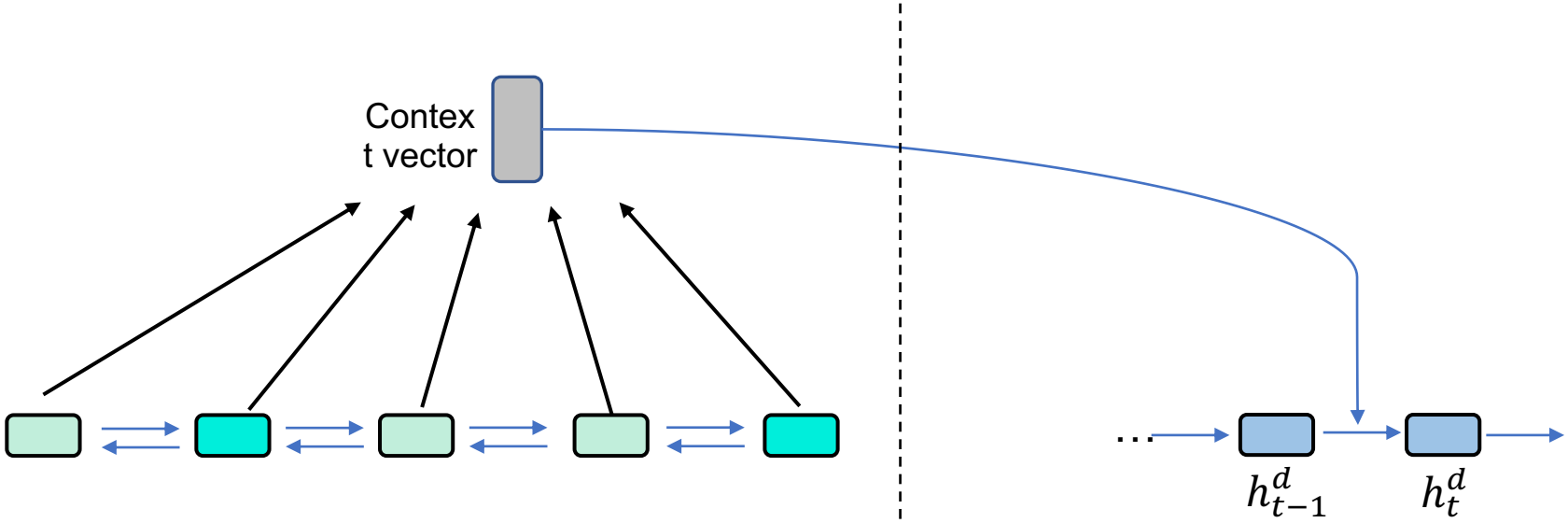
GitHub

Slido
NLP0316
(Link)

現代自然語言處理基本功

- Week 4 – Week 5
  - Sub-word 分詞法, 注意力機制
  - Transformer 架構



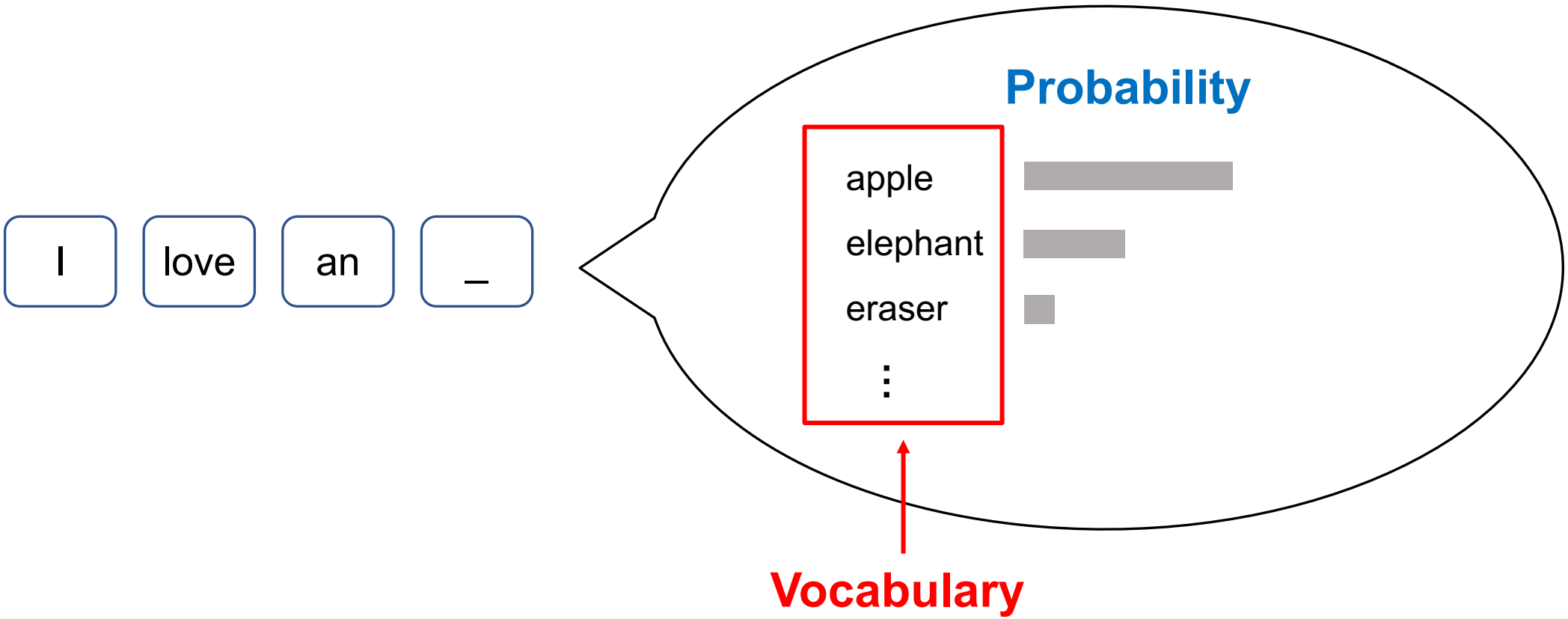


Outline

- Announcement, GAI money [10 min]
- Sub-word tokenization
  - Byte-pair Encoding (BPE) [30 min]
  - BPE Code [50 min]
- Attention (Not self-attention) [30 min]
- Quiz [20 min]

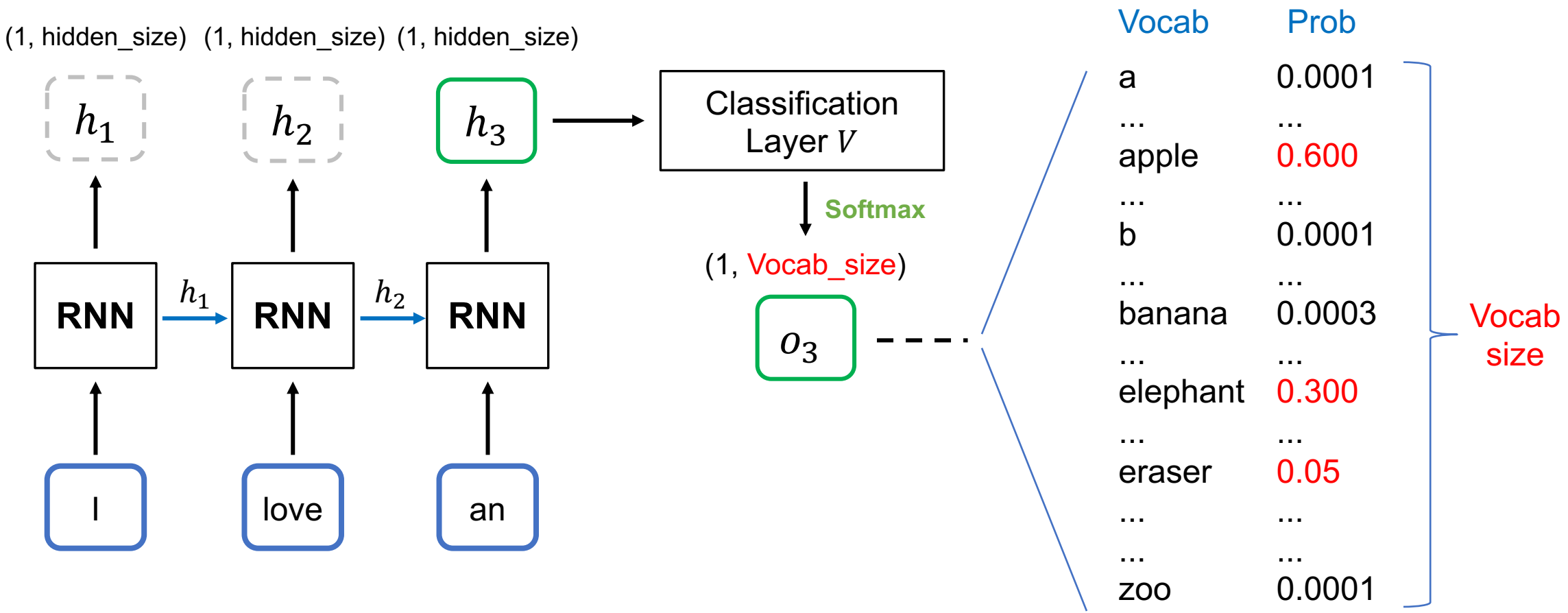
Sub-word Tokenization

Recap: Word Representations



Recap: Text Generation with RNN

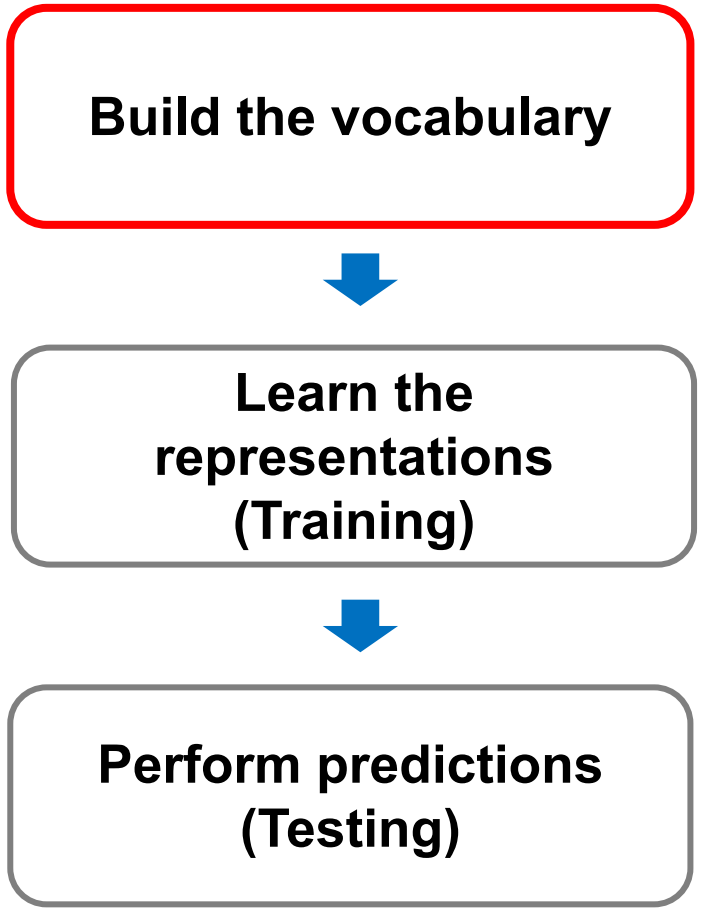
*本頁的RNN不包含輸出層V



NLP

Word embeddings

Basic Pipeline of Natural Language Processing



[Recap] How to build the vocabulary?

- Segment words based on delimiters (e.g., white spaces).
- Then we can collect the words from training corpora.

I love apples. I like apples and pineapples.

Vocabulary

Word	ID
and	0
apples	1
I	2
like	3
love	4
pineapples	5
.	6
<UNK>	7

Unknown words (not shown up in the training corpora)

(You can also perform stemming / lemmatization for building the vocabulary!)

Stemming (詞幹提取)

💡Stemming 是一種文字前處理方法，

目的是把不同變化型的單字，裁切成共同字根（stem），以減少詞彙數量。

Words	Stemmed results
connect, connected, connecting, connection	connect
study, studies, studied	studi

- https://cloud.google.com/discover/what-is-stemming
- https://www.nltk.org/howto/stem.html

Lemmatization (詞形還原)

💡類似於 stemming，lemmatization 也是一種文字前處理方法，

目的是把單字還原成它的字典原型 (lemma)，把不同詞形統一成同一個基本形式。

不同於 stemming， lemmatization 在處理字詞時會考慮到詞性和語言規則

Words	Lemma
connect, connected, connecting, connection	connect
study, studies, studied	study
running, ran	run
The autumn **leaves** are falling.	leaf (名詞)
He **leaves** the office at 6 PM.	leave (動詞)

https://www.nltk.org/api/nltk.stem.WordNetLemmatizer.html

Segmentation vs. Tokenization

- All tokenization is segmentation, but not all segmentation is tokenization.
- Segmentation can be:
  - Word segmentation from a sentence
  - Sentence segmentation from a document
- Tokenization can be:
  - Word segmentation from a sentence (then `words` become `tokens`)
  - Sub-word tokenization from a word or sentence

Delimiter-based Segmentation

NLP is fun, useful, and practical. ->

- NLP
- is
- fun,
- useful,
- and
- practical.

- **西方語言是用 (e.g., 英語)**
  - Cannot work for Chinese, Japanese.
- 模型訓練完成後，**無法處理不在字典中的字 (這些字通常是因為不存在於訓練集)**
  - 此情況也包含拼錯字的情形

[Recap] Embedding Lookup

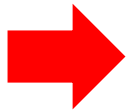
Index	Vocab	Dimension 1	Dimension 2
0	I	-0.4703061747695665 4	-4.440892098500626e-16
1	love	-8.326672684688674e-16	0.4464710977801294
2	do	8.881784197001252e-16	0.7838949952895327
3	deep	-0.4915010045415975	-1.3016463110087907e-16
4	learning	-1.582067810090848e-15	0.4314767609119055
5	NLP	-0.2523320838384413	-2.1837832516159107e-16
6	programming	-0.6881623238553303	-1.9142202344347 39e-16

Delimiter-based Segmentation (Continued.)

- 特別在機器翻譯任務，source 和 target 語言不一定 1-to-1 對齊
  - target language 出現複合詞時，如果模型只用 word-level token 難以學到複合關係；改用 subword，模型比較能學到可重複運用的對應關係
- For example, sewage water treatment plant (English) ->

Abwasserbehandlungsanlage (German)

sewage water　treatment　plant/facility



Sub-word units are favored.

Summary

- 為什麼我們需要 Sub-word Tokenization?
  - 為了解決 OOV 的問題
    - 有些語言特別容易出現長單詞，例如 Abwasserbehandlungsanlage 這種可拆式複合字
    - 沒看過的字 (Unseen words)
  - 擴增字典大小，如果原本字典在 30,000–50,000 個字，那如果用 Sub-word 的方式建立字典就可以在原本的字典大小範圍表達更多的字

Common Sub-word Tokenization Algorithms

- Byte Pair Encoding (Sennrich et al., 2016)[1]
- WordPiece (Schuster and Nakajima, 2012)[2]
- Unigram Language Model (Kudo, 2018)[3]

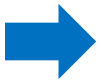
[1] Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units." Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL), 2016.
[2] Schuster, Mike, and Kaisuke Nakajima. "Japanese and korean voice search." 2012 IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, 2012.
[3] Kudo, Taku. "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates." Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL). 2018.

Sub-word Tokenization (1)

- Given a training corpus, we first turn each word into a sequence of characters (separated by spaces)

Training corpus

low low low low low lower lower newest newest newest newest newest newest widest widest widest



Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w e s t </w>	6
w i d e s t </w>	3

- </w> is a special end-of-word symbol, allowing us to restore the original tokenization.

Sub-word Tokenization (1)

- Initialize the vocabulary with the existing characters:

low low low low low lower lower newest newest
newest newest newest newest widest widest widest

Initial vocab: </w>, d, e, i, l, n, o, r, s, t, w

(逗號不算)

Sub-word Tokenization (2)

- Find the character **pair** with the highest frequency

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w e s t </w>	6
w i d e s t </w>	3

Found “e s” with the highest frequency
6+3=9

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w

Sub-word Tokenization (3)

- Add the pair with the highest frequency to the vocab

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w e s t </w>	6
w i d e s t </w>	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es

Sub-word Tokenization (4)

- Merge the characters by replacing all words in the corpus with the newly added pair.

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w **es** t </w>	6
w i d **es** t </w>	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es

Sub-word Tokenization (2)

Repeated (2)-(4)
according to `num_merges`

- Find the character **pair** with the highest frequency

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w es t </w>	6
w i d es t </w>	3

Found “es t” with the highest frequency
6+3=9

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es

Sub-word Tokenization (3)

Repeated (2)-(4)
according to `num_merges`

- Add the pair with the highest frequency to the vocab

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w es t </w>	6
w i d es t </w>	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est

Sub-word Tokenization (4)

Repeated (2)-(4)
according to `num_merges`

- Merge the characters by replacing all words in the corpus with the newly added pair.

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w **est** </w>	6
w i d **est** </w>	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est

Sub-word Tokenization (2)

Repeated (2)-(4)
according to `num_merges`

- Find the character **pair** with the highest frequency

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w est </w>	6
w i d est </w>	3

Found “est </w>” with the highest frequency
6+3=9

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est

Sub-word Tokenization (3)

Repeated (2)-(4)
according to `num_merges`

- Add the pair with the highest frequency to the vocab

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w est </w>	6
w i d est </w>	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>

Sub-word Tokenization (4)

Repeated (2)-(4)
according to `num_merges`

- Merge the characters by replacing all words in the corpus with the newly added pair.

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w **est</w>**	6
w i d **est</w>**	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>

Sub-word Tokenization (2)

Repeated (2)-(4)
according to `num_merges`

- Find the character **pair** with the highest frequency

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w est</w>	6
w i d est</w>	3

Found “l o” with the highest frequency 5+2=7

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>

Sub-word Tokenization (3)

Repeated (2)-(4)
according to `num_merges`

- Add the pair with the highest frequency to the vocab

Word	Frequency
l o w </w>	5
l o w e r </w>	2
n e w est</w>	6
w i d est</w>	3

Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>, lo

Sub-word Tokenization (4)

Repeated (2)-(4)
according to `num_merges`

- Merge the characters by replacing all words in the corpus with the newly added pair.

Word	Frequency
lo w </w>	5
lo w e r </w>	2
n e w est</w>	6
w i d est</w>	3

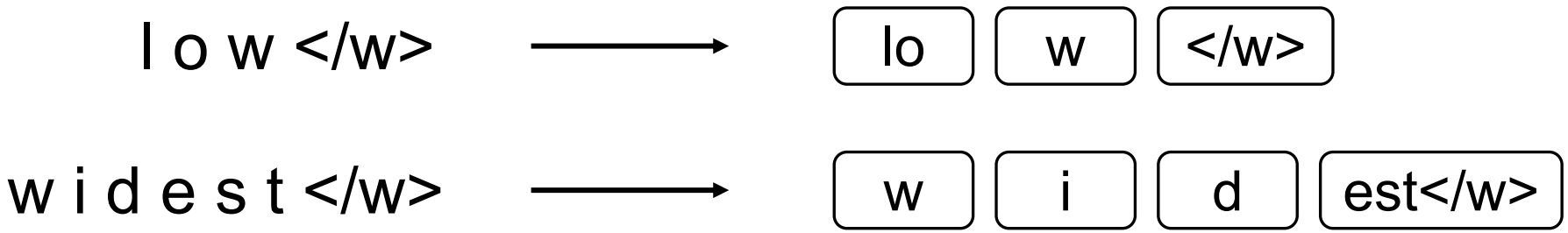
Current Vocabulary: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>, lo

Finishing the Sub-word Leaning

- Assume `num_merges`=4 (we just repeated four times.)
- `num_merges` is a hyperparameter that you need to set for BPE.
- The learned vocabulary is: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>, lo

Tokenization with Learned BPE

- The learned vocabulary is: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>, lo
- We first turn each word into a sequence of characters with </w> placed at the end of a word, same as the first step during the learning phase.
- Then we can merge the characters according to the learned vocabulary.
- Examples:



[注意事項] Tokenization with Learned BPE

- 合併的順序：left to right
- 在實際 tokenization 時，常會從左到右掃描字串，依照已學到的 merge rules 去合併
  - 每次合併的依據：依照 learned merge rules 進行；若可形成較長的已學習子詞，通常會繼續合併 (e.g., low: l+o→lo, lo+w→low)

Properties of BPE

- The final learned vocabulary size = **initial** size + `num_merges`

  Initial vocab: </w>, d, e, i, l, n, o, r, s, t, w

  Learned vocab: </w>, d, e, i, l, n, o, r, s, t, w, es, est, est</w>, lo

- This algorithm is based on statistics, so frequent sub-word units in provided corpora will be put to the learned vocabulary.

Attention

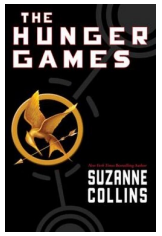
Attention relationship among words

Katniss looked at Prim sleeping beside her and felt a sudden heaviness, because today was the Reaping.

?

特殊節日

沈重

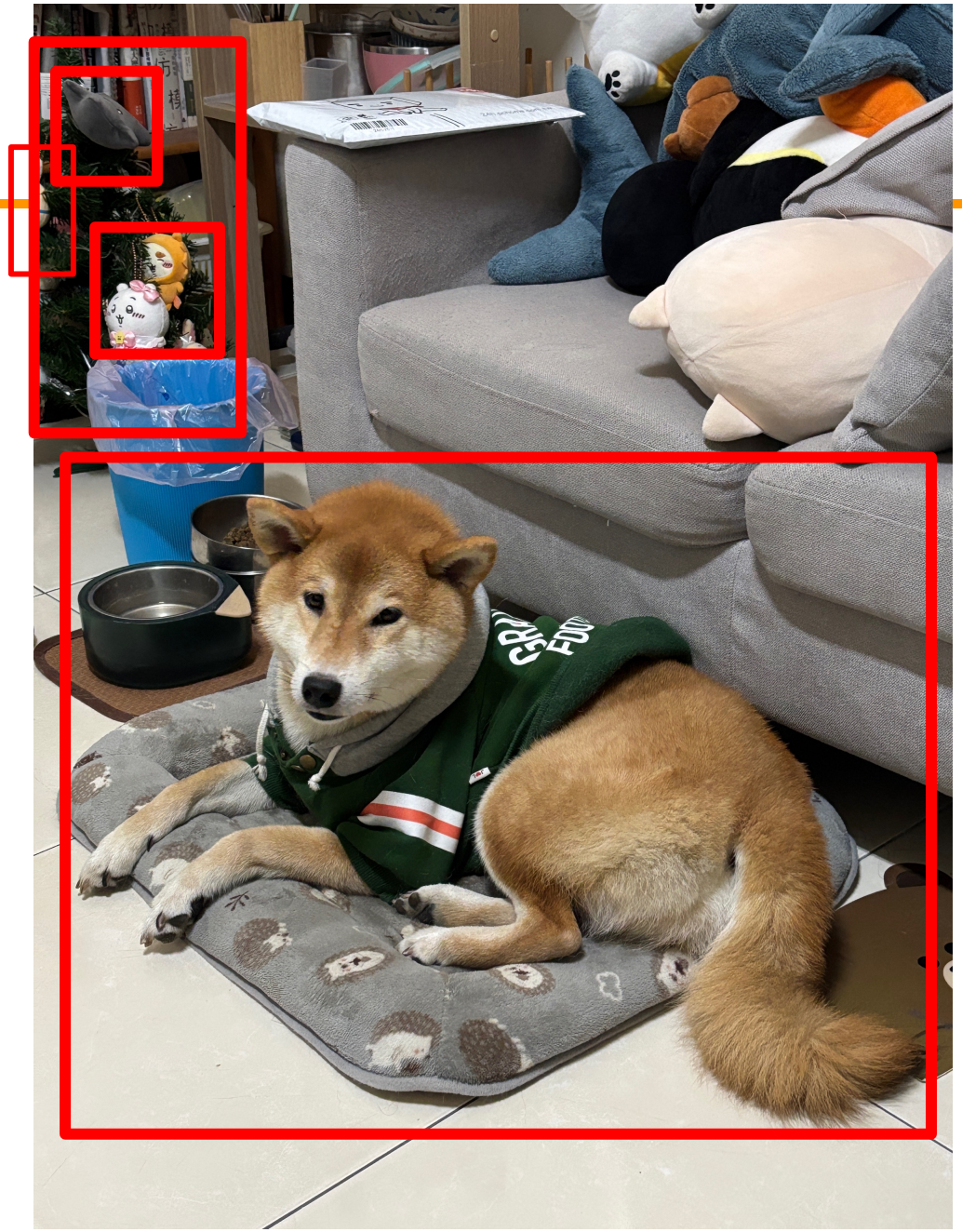


注意力

任務1：找出與時間相關的字詞

任務2：情感分類任務

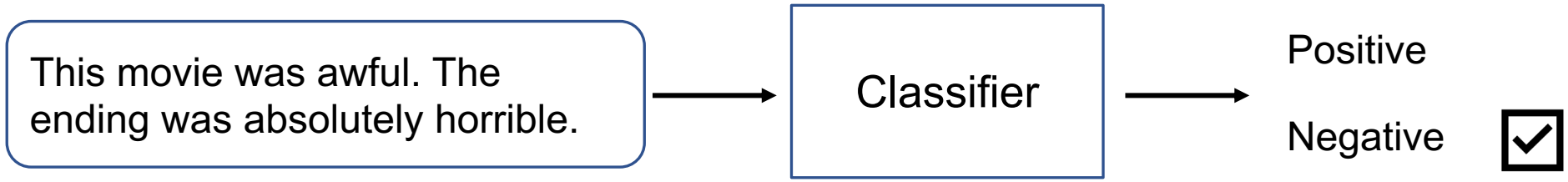
Attention

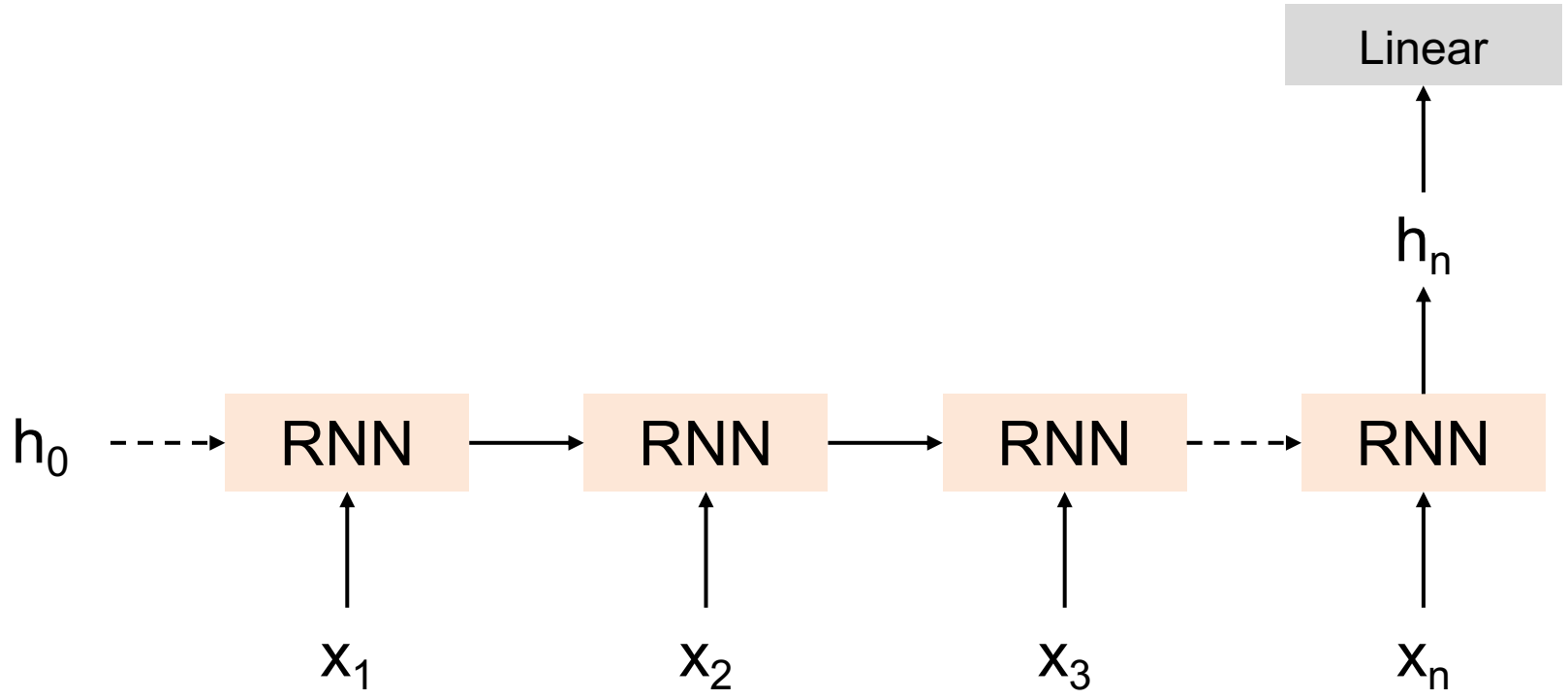


What is Attention?

💡 attention (注意力機制): 在理解一個輸入時，不平均看所有部分，而是根據當前任務，動態聚焦在更重要的位置。

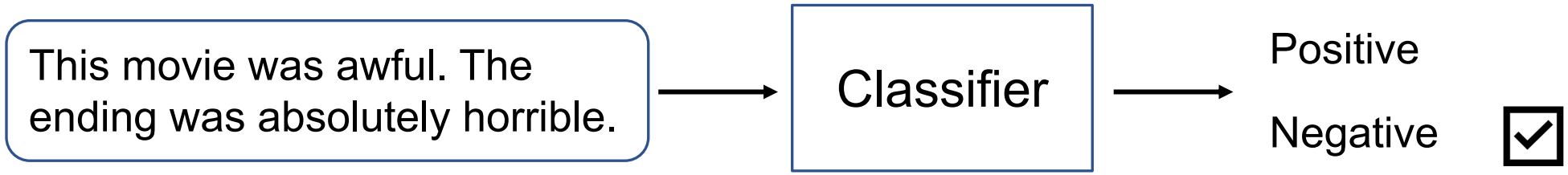
RNNs for Sequence Classification



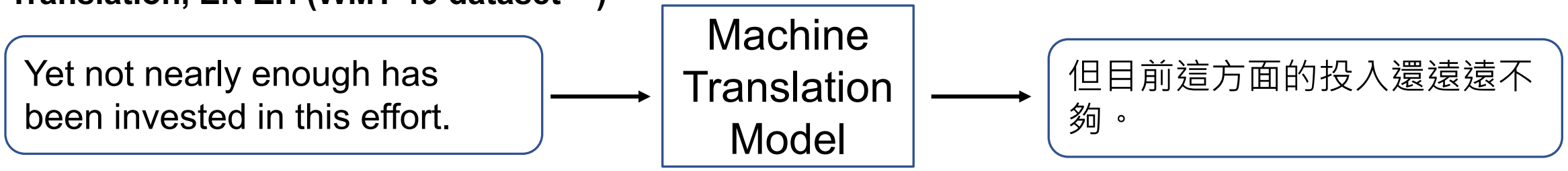


Machine Translation

Sentiment Analysis (IMDb dataset [1])



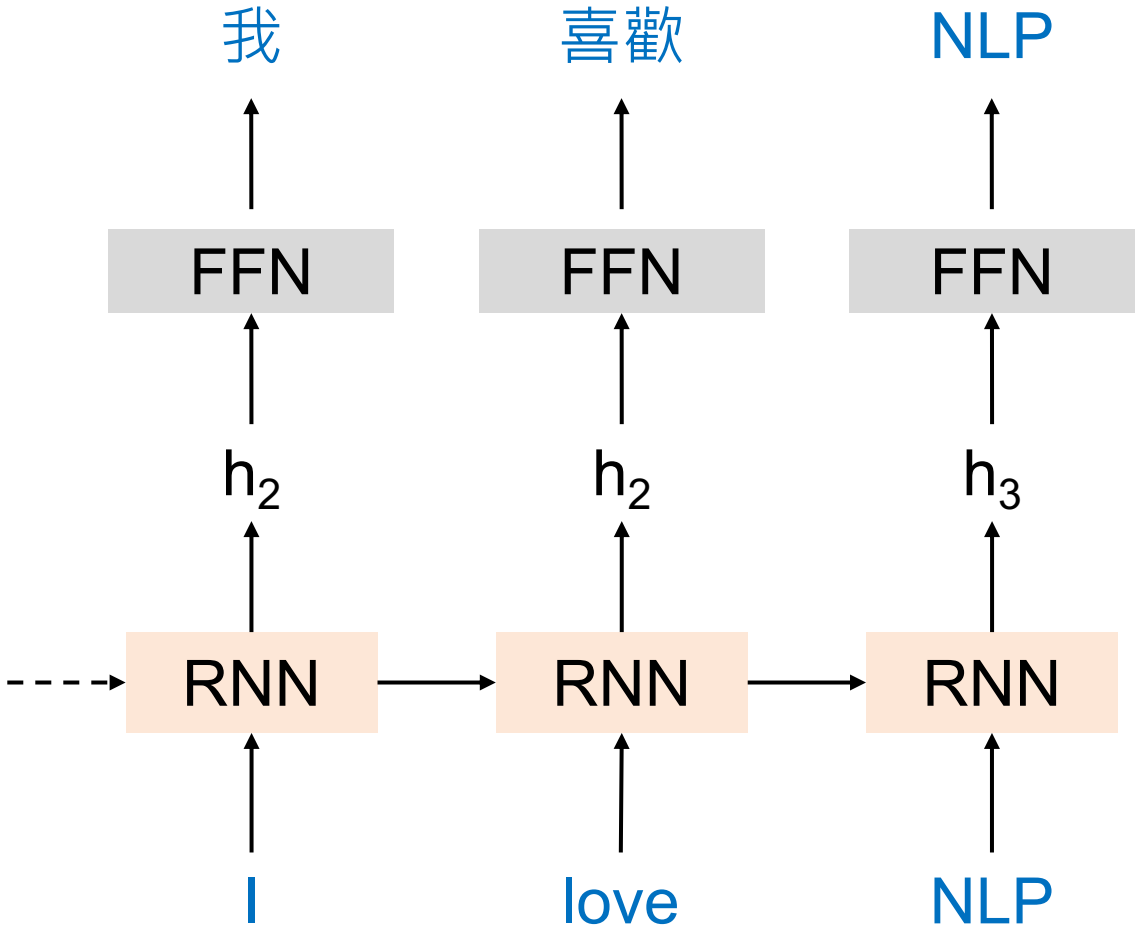
Translation, EN-ZH (WMT-19 dataset [2])



[1] https://huggingface.co/datasets/stanfordnlp/imdb
[2] https://huggingface.co/datasets/wmt/wmt19/viewer/zh-en

RNNs for Machine Translation?

Method 1 (逐字對齊)

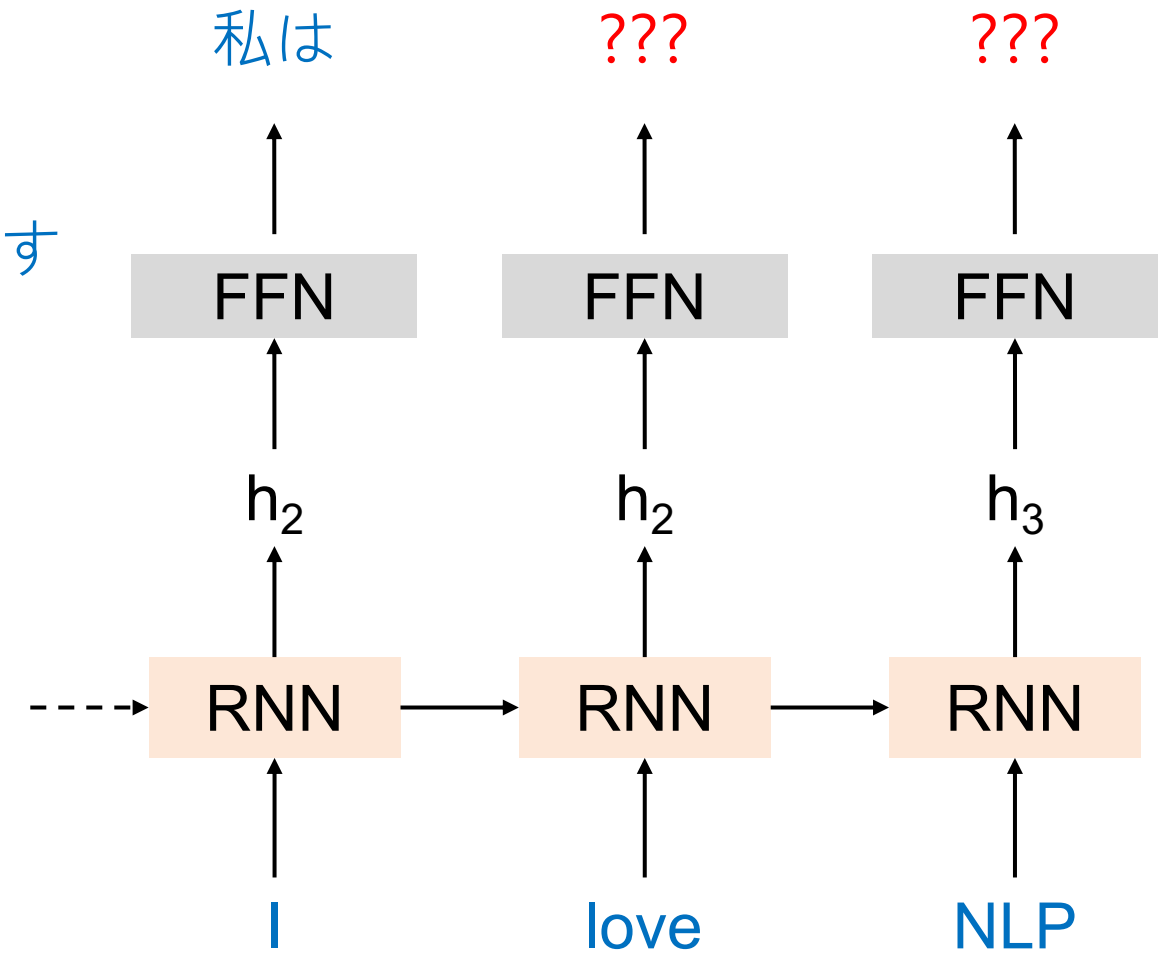


RNNs for Machine Translation?

Method 1 (逐字對齊)

Target sequence:

私は NLP が好きです

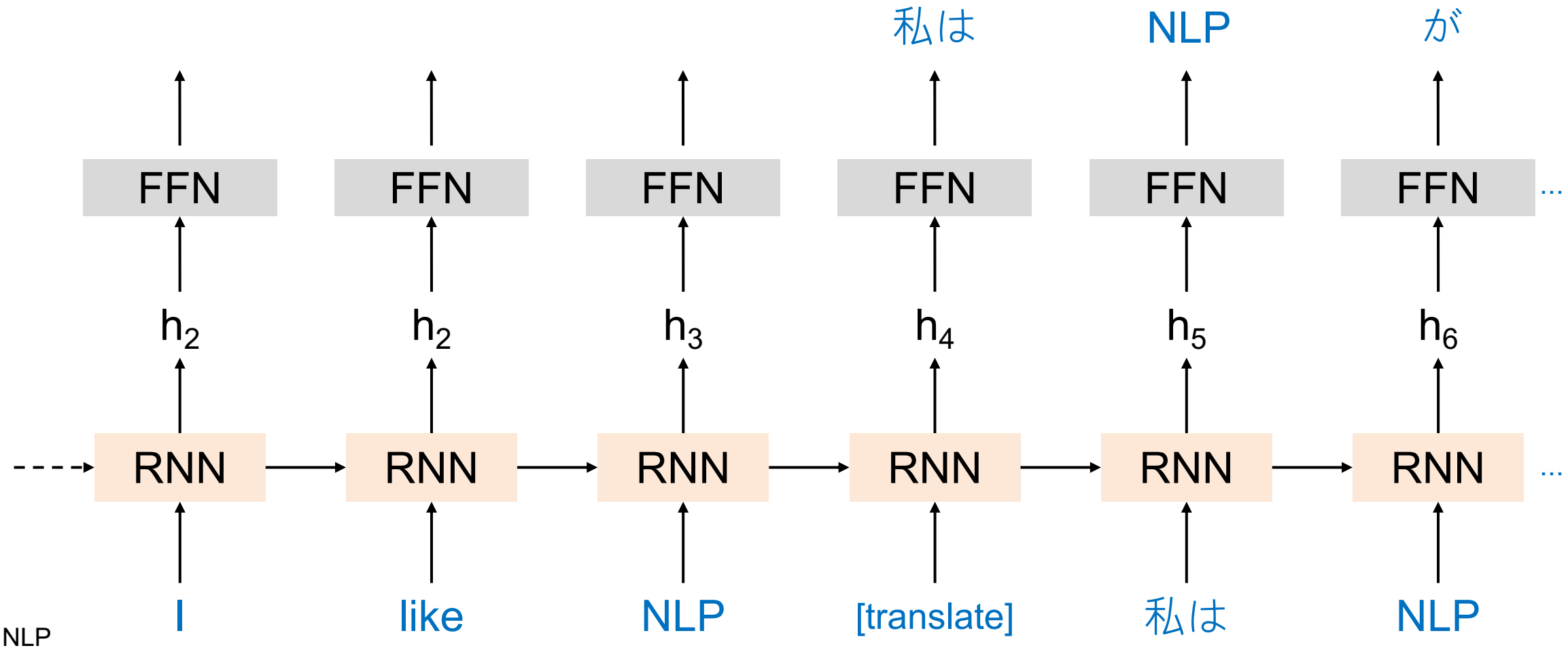


缺點：在不能逐字對齊的資料上效果不佳

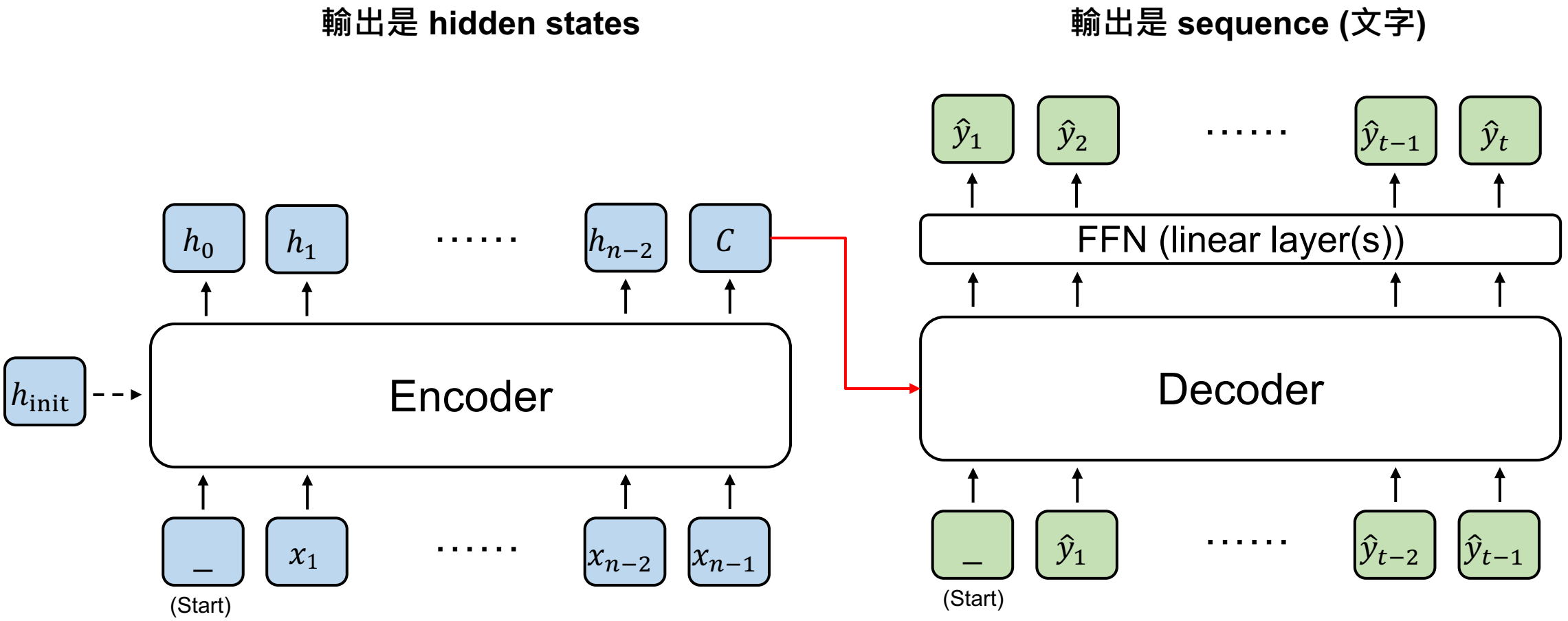
RNNs for Machine Translation?

缺點：長距離的關係難以被記憶

Method 2 (利用特殊 token)



Encoder and Decoder



(Encoder 的輸出有很多種
被Decoder運用的方式)

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

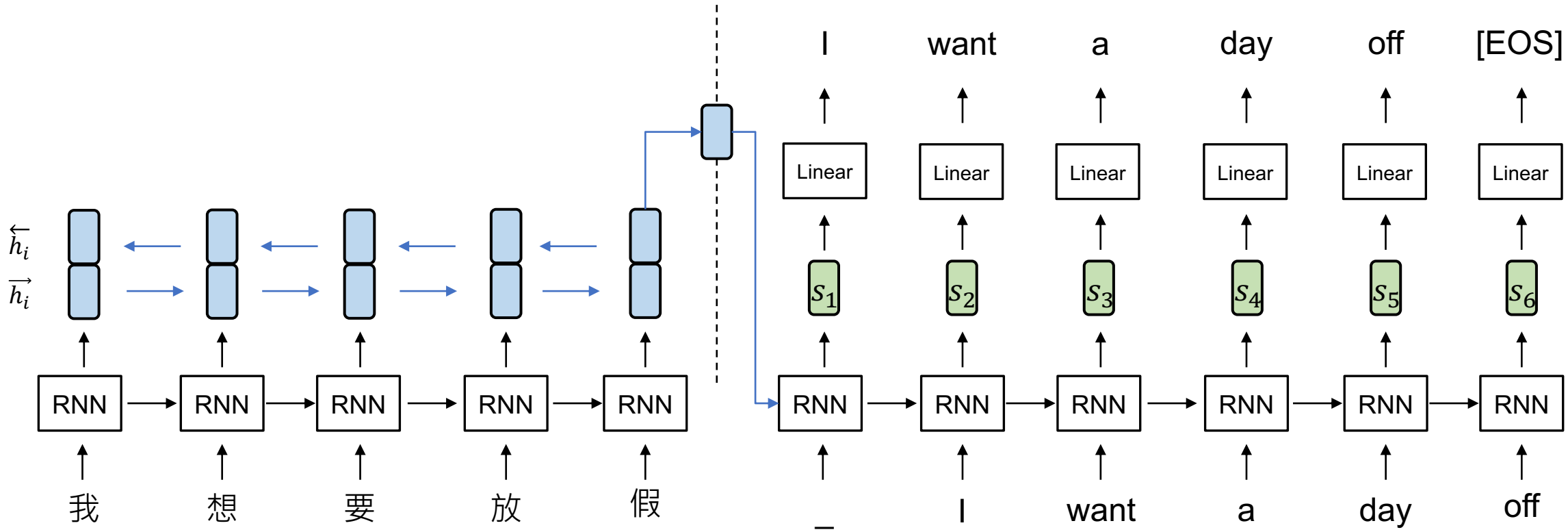
Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

機器翻譯 (一般情況)

i: encoder timestep
t: decoder timestep

$$\overrightarrow{h_i} = \left[\overrightarrow{h_i}; \overleftarrow{h_i}\right]$$



一般情況的 encoder-decoder 有什麼問題？

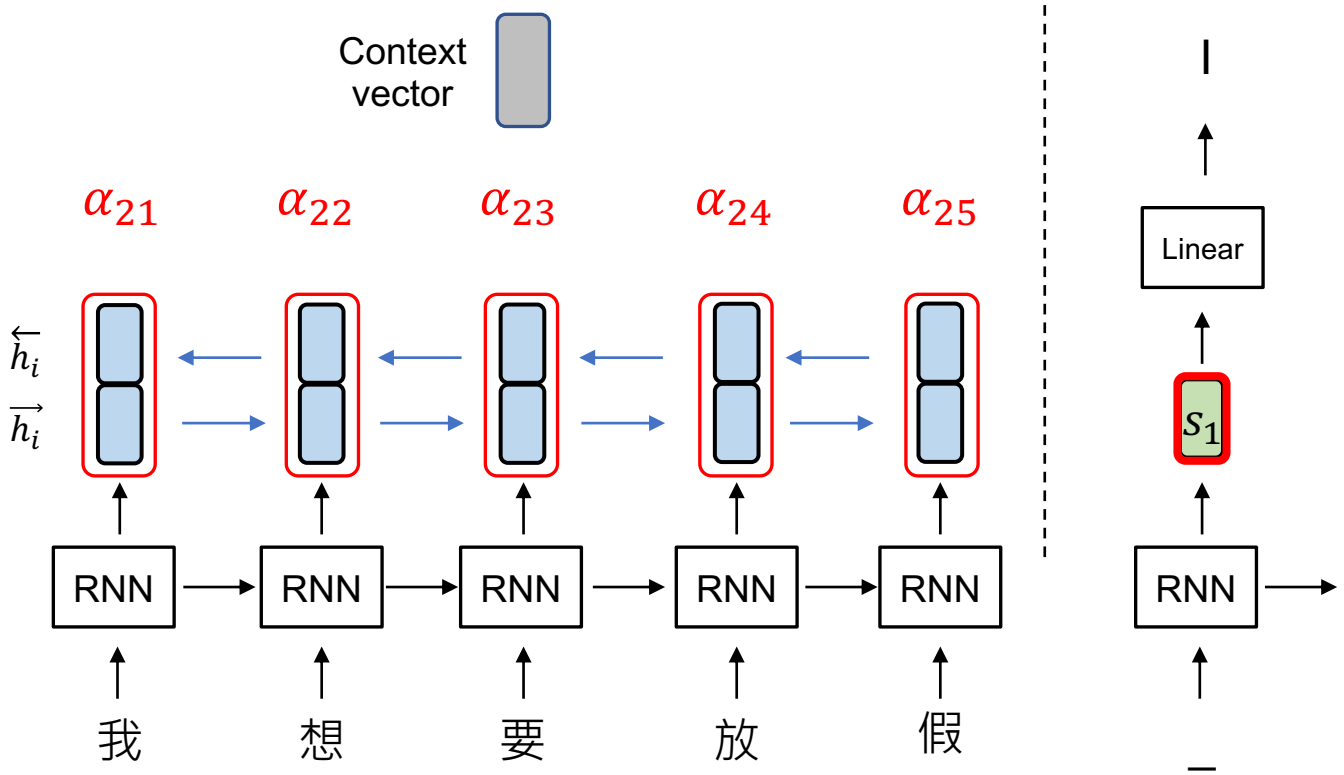
- 依賴 encoder 的 hidden state
  - 好像要你看完一整篇文章後，不能再回頭看原文，只能憑腦中最後留下的一小段印象把它完整翻譯出來
- 不管原句多短多長，最後都要塞進同樣大小的向量

機器翻譯 (一般情況, t = 2)

$\overrightarrow{h_i} = \left[\overrightarrow{h_i}; \overleftarrow{h_i}\right]$

$$c_2 = \sum_{i=1}^{n} \alpha_{2i} h_t$$

i: encoder timestep
t: decoder timestep

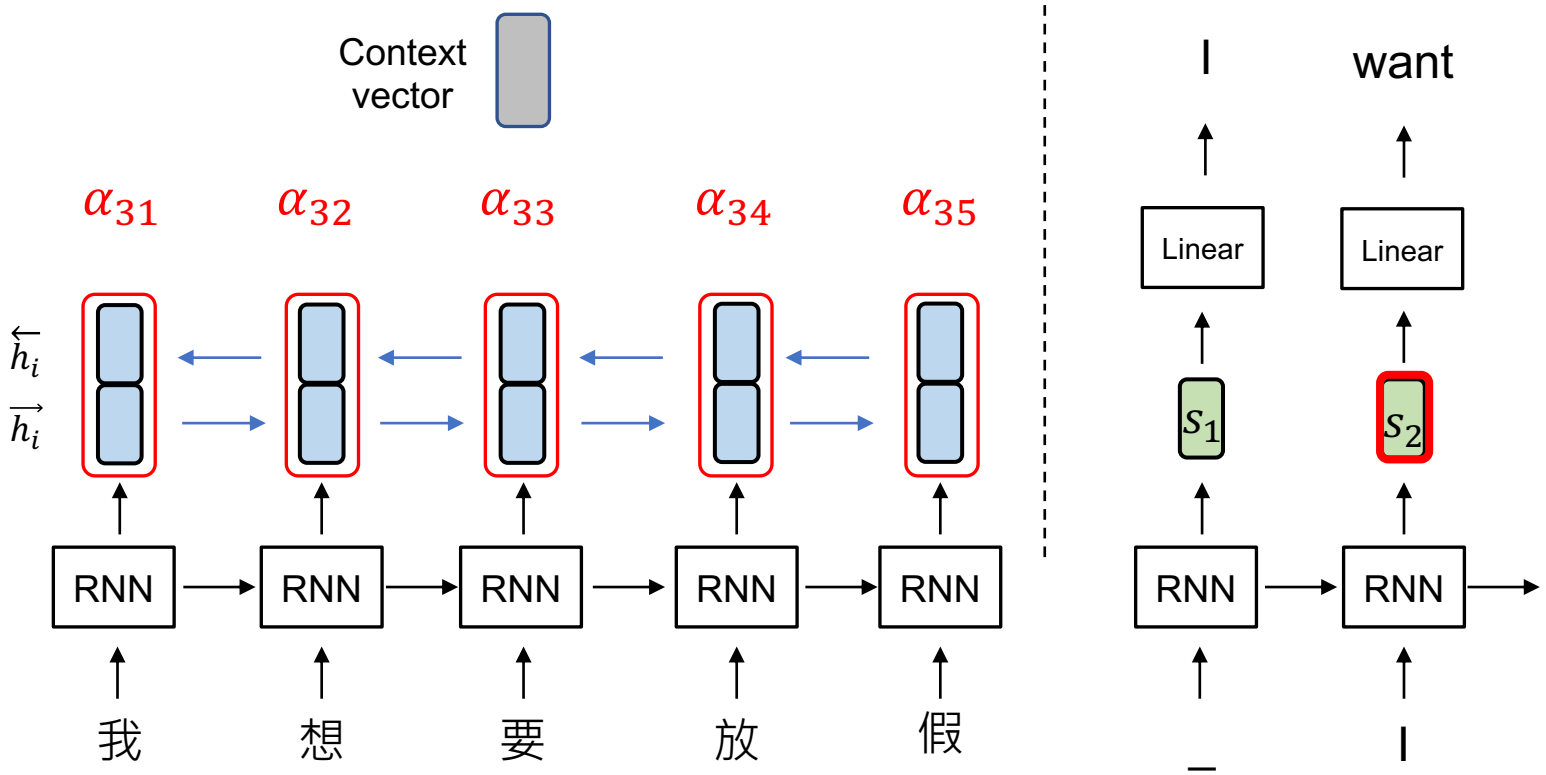


機器翻譯 (一般情況, t = 3)

$\overrightarrow{h_i} = \left[\overrightarrow{h_i}; \overleftarrow{h_i}\right]$

$$c_3 = \sum_{i=1}^{n} \alpha_{3i} h_t$$

i: encoder timestep
t: decoder timestep

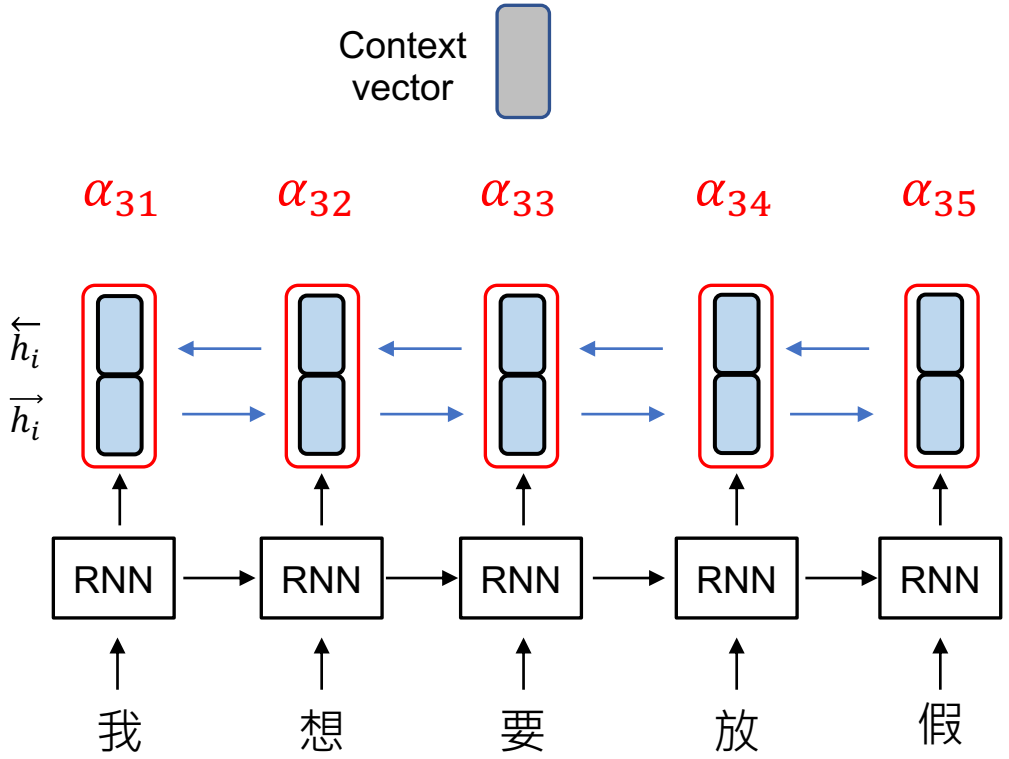


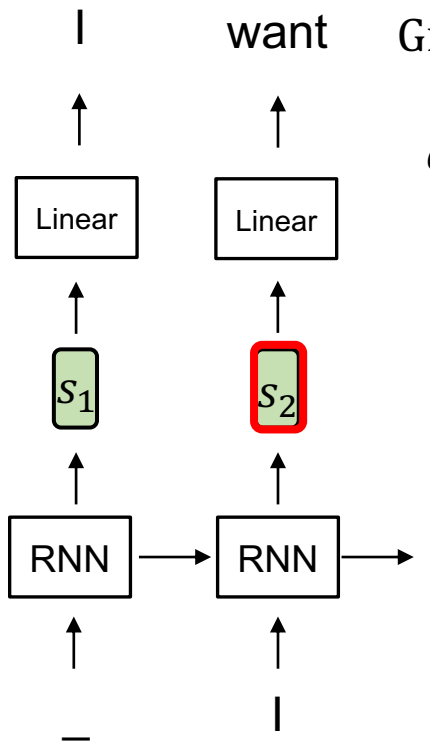
Additive Attention (算出 α_{ti})

$\overrightarrow{h_i} = \left[\overrightarrow{h_i}; \overleftarrow{h_i}\right]$

$$c_3 = \sum_{i=1}^{n} \alpha_{3i} h_t$$

i: encoder timestep
t: decoder timestep





Given i ∈ {1, ..., n},

$$\alpha_{ti} = \text{softmax}(v \cdot \tanh(W s_{t-1} + U h_i))$$

Additive Attention

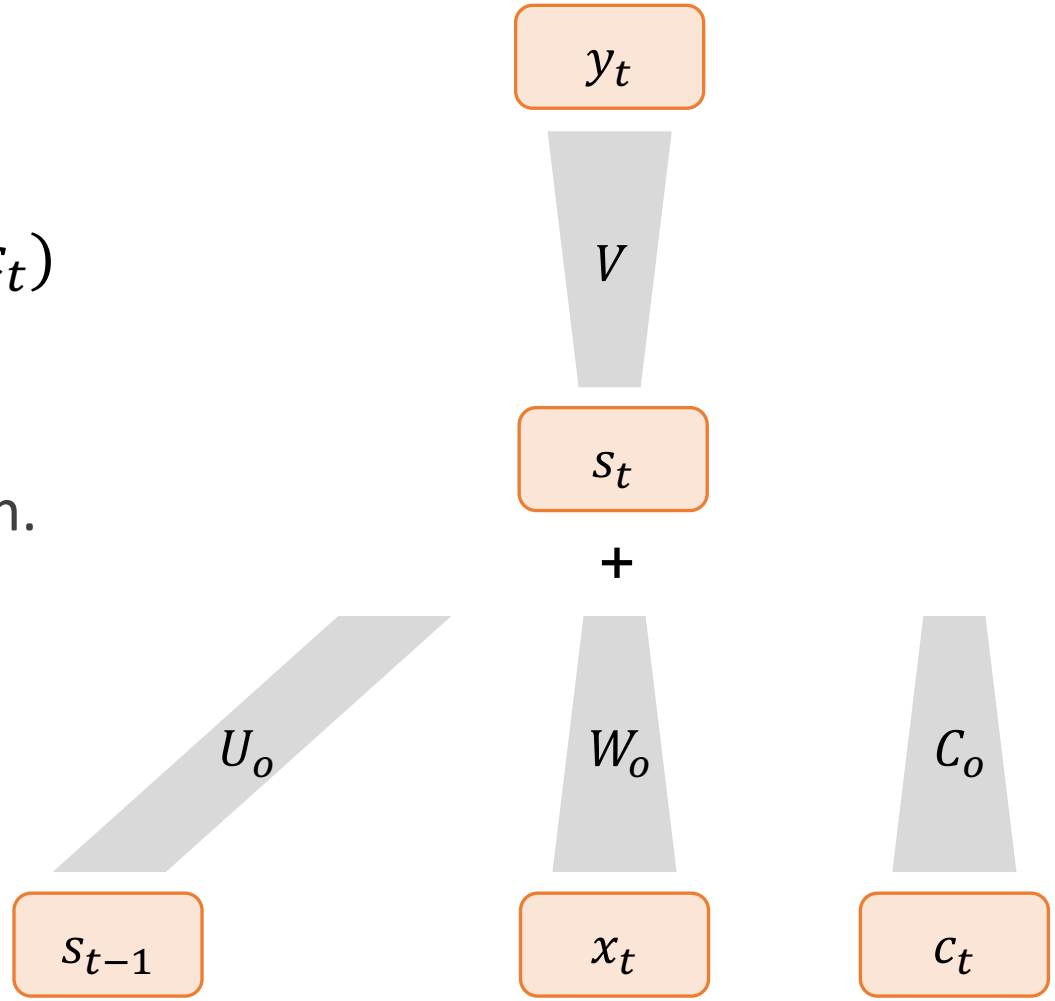
轉為1個數字

得到了 Context vector 之後

- The equation on step t is:

$$s_t = \tanh(U_o s_{t-1} + W_o x_t + C_o c_t)$$
$$y_t = \text{softmax}(V s_t)$$

, where tanh is an activation function.



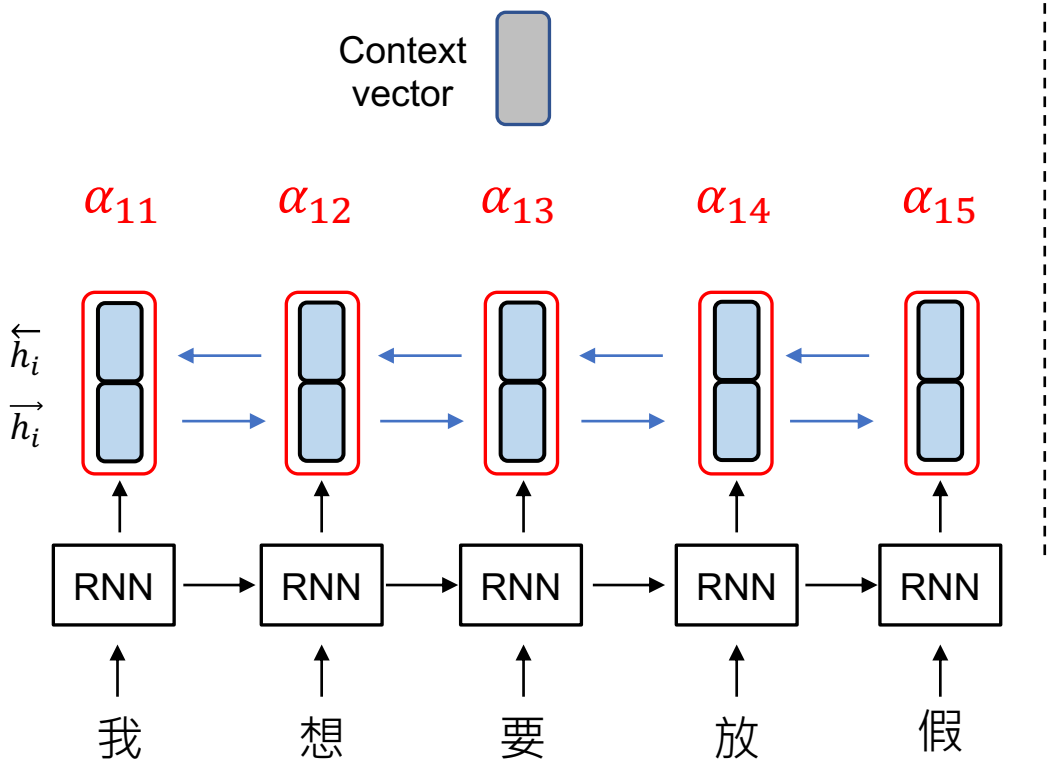
爲方便同學理解，此頁為 Bahdanau et al. (2014). 的模型簡化版

機器翻譯 (一般情況, t = 1)

$\overrightarrow{h_i} = \left[\overrightarrow{h_i}; \overleftarrow{h_i}\right]$

$$c_1 = \sum_{i=1}^{n} \alpha_{1i} h_t$$

i: encoder timestep
t: decoder timestep



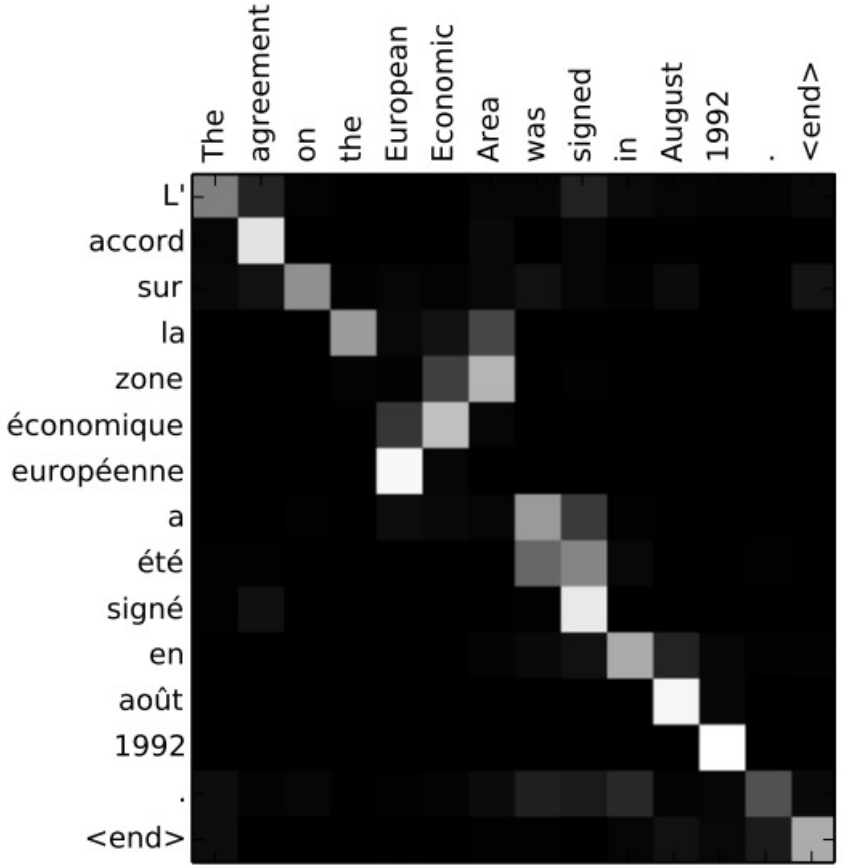
Given i ∈ {1, ..., n},

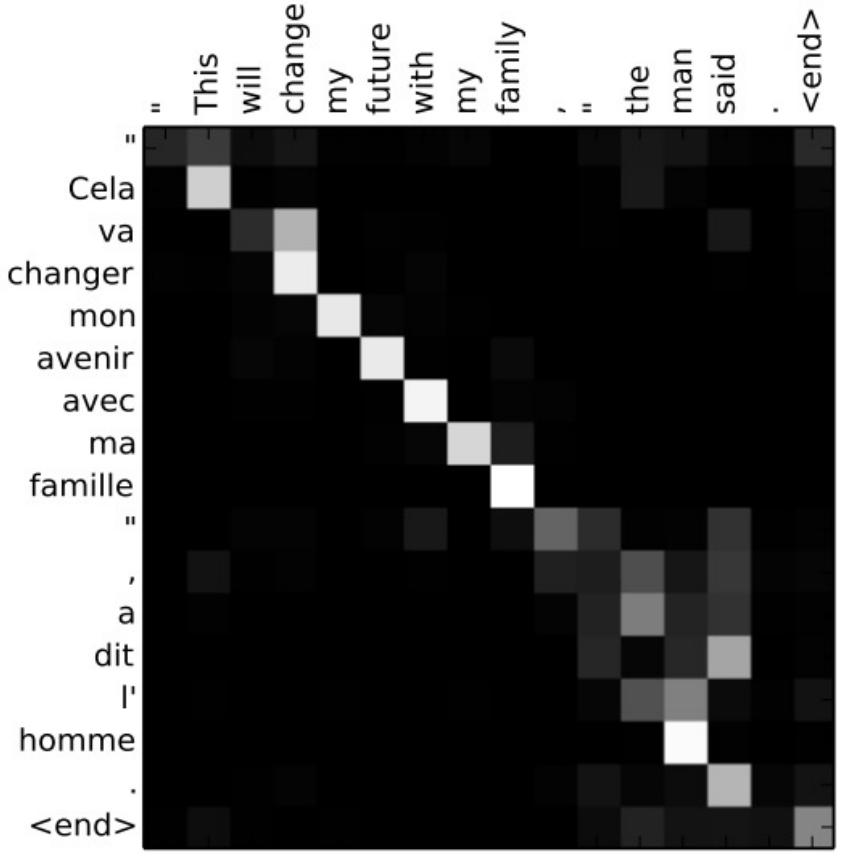
$$\alpha_{1i} = \text{softmax}(v \cdot \tanh(W s_0 + U h_i))$$

$s_0 = \tanh(W_s \overleftarrow{h_1})$

Attention

An example of machine translation.





Other Attention Approaches

Dot-product Attention: https://arxiv.org/abs/1508.04025

https://lilianweng.github.io/posts/2018-06-24-attention/

Thank you!

Instructor: 林英嘉

✉ yjlin@cgu.edu.tw

TA: 林宣辰

✉ b1128015@cgu.edu.tw